Algoritmo e Laboratório de Programação II

Aula 2 – Matrizes – Parte 2 (Nível Avançado)

Roteiro

- Strings
- As funções para manipulação de Strings
- Aritmética com endereços
- Outras funções para manipulação de Strings: strlen(); strcat(); strcmp(); strcpy(); strncat(); srtncmp(); strncpy()
- Matrizes de Strings.
- Um programa que imprime um cartão de Natal

Strings

- O uso mais importante de matrizes é aplicado à criação de tipos de dados para armazenar e manipular textos como palavras, nomes e sentenças.
- String é uma matriz do tipo char, que armazena um texto formado de caracteres e sempre terminado pelo caractere zero (‘0’).
- Em outras palavras, string é uma série de caracteres, em que cada um ocupa um byte de memória, armazenando em sequência e terminando por um byte de valor zero (‘0’).

Strings

- Cada caractere é um elemento independente da matriz e pode ser acessado por meio de um índice.

Strings Constantes

- Sempre que o compilador encontra qualquer coisa entre aspas duplas, ele reconhece que se trata de uma string constante.
- Ao longo do curso básico, já mostramos vários exemplos de strings constantes.
- Veja mais um exemplo:

```
printf("%s","Saudações");
```

Strings Constantes

- Na memória a cadeia de caracteres “Saudações!” é armazenada da seguinte forma:

Endereço	Conteúdo
1449	
1450	S
1451	a
1452	u
1453	d
1454	a
1455	ç
1456	õ
1457	e
1458	s
1459	!
1460	\0
1461	

Strings Constantes

- Observe que o caractere ‘\0’, também chamado NULL, tem valor zero decimal, e não é o mesmo que o caractere ‘\0’, com valor 48 decimal.
- A terminação com zero é importante, pois é a única maneira de as funções reconhecerem onde é o fim da string.

Strings Variáveis

- Um dos modos de se les strings do teclado é por meio da função scanf(), pelo formato %s.
- O exemplo seguinte mostra como definir uma string variável para receber um nome pelo teclado.

Strings Variáveis

Prog13.c

```c
/* Prog13.c */
/* Mostra o uso de strings */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char nome[80];
    printf("Digite o seu nome: ");
    scanf("%s", nome);
    printf("Saudações %s.\n", nome);
    system("PAUSE");
    return 0;
}
```

Roteiro

- Strings
- As funções para manipulação de Strings
- Aritmética com endereços
- Outras funções para manipulação de Strings: strlen(); strcat(); strcmp(); strcpy(); strncat(); srtncmp(); strncpy()
- Matrizes de Strings.
- Um programa que imprime um cartão de Natal

As Funções para Manipulação de Strings

- A função scanf():
  - A instrução

```
scanf("%s", nome);
```

  - Lê cada caractere digitado e os armazena a partir do endereço nome.
  - O processo termina quando um caractere branco é encontrado.
  - Nesse ponto é incluído automaticamente o caractere '\0' na próxima posição livre da matriz.
  - Observe que você não poderá ultrapassar o limite estipulado na criação da matriz e deverá prever espaço para o caractere '\0'.

As funções para Manipulação de Strings

- A função scanf() entende um espaço em branco como término da entrada.
- O resultado é que não existe uma forma de digitar um texto de múltiplas palavras.
- Assim, somente o primeiro nome foi armazenado.
- Você deve ter notado que não usamos o operador & precedendo o segundo argumento de scanf().
- Já que o nome de uma matriz é o seu endereço inicial, seria errado utilizar o operador de endereço junto a ele.

As funções para Manipulação de Strings

- A expressão nome é equivalente a &nome[0].
- A função scanf() é principalmente usada para ler uma mistura de tipos de dados numa mesma instrução.
- Por exemplo, se em cada linha de entrada quisermos ler o código de um material do estoque, o seu número no estoque e o seu preço, scanf() se adapta perfeitamente.

As funções para Manipulação de Strings

- A função gets():
  - A função gets(), como prtótipo no arquivo stdio.h, é mais conveniente para a leitura de textos.
  - O seu protótipo é unicamente ler uma cadeia de caracteres do teclado enquanto não for pressionada a tecla [ENTER].
  - Todos os caracteres são armazenados na string e é incluído o caractere NULL no final.
  - Caracteres brancos, como espaços e tabulações, são perfeitamente aceitáveis como parte da cadeia. Veja o programa modificado:

As funções para Manipulação de Strings

Prog14.c

```c
/* Prog14.c */
/* Mostra leitura de string com gets() */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char nome[80];
    printf("Digite o seu nome: ");
    gets(nome); /* lê texto do teclado */
    printf("Saudações %s.\n", nome);
    system("PAUSE");
    return 0;
}
```

As funções para Manipulação de Strings

- Impressão de strings com puts():
  - A função puts() é o complemento da função gets e é bem fácil de usar.
  - O propósito de puts() é imprimir uma única string por vez.
  - O endereço dessa string deve ser enviado para puts() como argumento.
  - O próximo exemplo ilustra seu uso:

As funções para Manipulação de Strings

Prog15.c

```c
/* Prog15.c */
/* Mostra impressão de string com puts() */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char nome[80];
    printf("Digite o seu nome: ");
    gets(nome); /* lê texto do teclado */
    puts("Saudacoes, ");/* imprime texto no vídeo */
    puts(nome);
    puts("puts() pula de linha sozinha");
    puts(&nome[4]);
    system("PAUSE");
    return 0;
}
```

As funções para Manipulação de Strings

- Impressão de strings com puts():
  - A função puts() reconhece o '\0' como o fim da string.
  - Observe que cada string impressa por puts() termina por um caractere de nova linha.
  - Portanto, se você desejar imprimir duas strings na mesma linha, a solução é usar printf().
  - As duas instruções seguintes tem o mesmo efeito:

```
printf("%s\n", nome);
puts(nome);
```

Inicialização de Strings

- Lembre-se de que qualquer matriz pode ser inicializada, contanto que seja da classe extern ou static.
- Essa característica vale também para string que são matrizes do tipo char.

```
char nome[] = {'A', 'n', 'a', '\0'};
```

- O compilador oferece uma forma de inicialização equivalente, mas consideravelmente mais simples:

```
char nome[] = "Anna";
```

Inicialização de Strings

Prog16.c

```
1   /* Prog16.c */
2   /* Mostra o uso de strings inicializadas */
3   #include <stdio.h>
4   #include <stdlib.h>
5
6   int main()
7   {
8       char salute[]="Saudações, ";
9       char nome[80];
10      printf("Digite o seu nome: ");
11      gets(nome);
12      printf("%s%s\n", salute, nome);
13      system("PAUSE");
14      return 0;
15  }
```

Roteiro

- Strings
- As funções para manipulação de Strings
- Aritmética com endereços
- Outras funções para manipulação de Strings: strlen(); strcat(); strcmp(); strcpy(); strncat(); srtncmp(); strncpy()
- Matrizes de Strings.
- Um programa que imprime um cartão de Natal

Aritmética com Endereços

- Já sabemos que o nome de uma matriz é um endereço, que corresponde ao endereço do primeiro elemento da matriz.
- Em outras palavras, salute é o endereço de salute[0], que é uma variável do tipo char.
- Ou seja, salute é o endereço de uma variável do tipo char.

Aritmética com Endereços

- O que significa salute + 1?
- O número 1 desta soma tem uma unidade estranha:
  - Significa um byte, se o endereço for de uma variável char;
  - 4 bytes (ou tamanho da palavra da máquina), se o endereço for de uma variável int;
  - 4 bytes, se o endereço for de uma variável float;
  - E assim por diante.

Aritmética com Endereços

- Portanto, se somarmos 1 ao endereço de uma matriz de elementos do tipo int, estaremos obtendo o endereço da proxima variável int da memória, ou o proximo elemento da matriz.
- Em regra geral, se M é o nome de uma matriz e i é uma variável do tipo int, então M + i é equivalente a M[i].

Aritmética com Endereços

- Utilizando esse conceito, podemos extrair uma substring de uma string somando ao seu endereço um número inteiro que indique o byte onde começará nossa substring.
- Veja o exemplo:

Aritmética com Endereços

Prog17.c

```c
/* Prog17.c */
/* Mostra o uso da aritmética com endereços */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char salute[]="Saudações, ";
    char nome[80];
    printf("Digite o seu nome: ");
    gets(nome);
    printf("%s%s\n", salute, nome + 5);
    system("PAUSE");
    return 0;
}
```

Roteiro

- Strings
- As funções para manipulação de Strings
- Aritmética com endereços
- Outras funções para manipulação de Strings: strlen(); strcat(); strcmp(); strcpy(); strncat(); srtncmp(); strncpy()
- Matrizes de Strings.
- Um programa que imprime um cartão de Natal

Outras funções de manipulação de Strings

- Os compiladores oferecem, em suas bibliotecas, várias funções para trabalhar com strings.
- A seguir analisaremos algumas delas.

Outras funções de manipulação de Strings

- A Função strlen():
  - A função strlen(), declarada no arquivo string.h, recebe como argumento o endereço de uma string e retorna o seu tamanho (o número de caracteres armazenados na string).
  - O programa seguinte examina cada posição de memória ocupada pela string e imprime o que tiver encontrado.

Outras funções de manipulação de Strings

Prog18.c

```c
/* Prog18.c */
/* Mostra o uso de strlen() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int len, i;
    char nome[80];

    printf("Digite o seu nome: ");
    gets(nome);

    len=strlen(nome);

    for(i=0; i<len + 4; i++)
        printf("Endereço = %u\tChar  = %c\tDec   = %d\n",
               (unsigned)(nome+i), nome[i], (int)nome[i]);

    system("PAUSE");
    return 0;
}
```

Outras funções de manipulação de Strings

- A Função strlen():
  - Imprimindo quatro caracteres além do final da string.
  - O primeiro é o NULL, que não aparece, pois tem valor zero; já os outros são caracteres considerados lixo.
  - Em nosso programa a instrução:

```
len=strlen(nome);
```

  - Atribui o valor 5 à variável len se o nome “Andre” for inserido. Observe que strlen() não conta o caractere NULL.

Outras funções de manipulação de Strings

- A Função strcat():
  - A função strcat(), declarada no arquivo string.h, concatena uma cadeia de caracteres a outra, ou seja, acrescenta uma cadeia de variáveis do tipo char e concatena a segunda na primeira.
  - A segunda cadeia não é alterada.
  - Cuidado! Essa função não verifica se a primeira cadeia tem espaço suficiente para que a segunda cadeia seja adicionada ao seu final.

Outras funções de manipulação de Strings

Prog19.c

```c
/* Prog19.c */
/* Mostra o uso de strcat() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char salute[100]="Saudações, ";
    char nome[80];

    printf("Digite o seu nome: ");
    gets(nome);

    strcat(salute, nome);

    printf("%s\n", salute);

    system("PAUSE");
    return 0;
}
```

Outras funções de manipulação de Strings

- A Função strcmp():
  - Suponhamos que você queira comparar uma cadeia de caracteres digitada pelo usuário com alguma string interna do programa e decida escrever o seguinte programa:

Outras funções de manipulação de Strings

Prog20.c

```c
/* Prog20.c */
/* Mostra a comparação entre strings */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char resposta[]="BRANCO";
    char resp[40];

    printf("Qual é a cor do cavalo branco de Napoleão?");
    gets(resp);

    while (resp != resposta)
    {
        printf("Resposta errada. Tente de novo.\n");
        gets(resp);
    }
    printf("Correto!\n");
    system("PAUSE");
    return 0;
}
```

Outras funções de manipulação de Strings

- A Função strcmp():
  - Este programa não trabalhará corretamente, pois resp e resposta são endreços de variáveis char, então a expressão:

```
while (resp != resposta)
```

  - Realmente não pergunta se as duas cadeias são diferentes e sim se os dois endereços são diferentes.
  - Como resp e resposta são endereços distintos, o laço while é infinito.

Outras funções de manipulação de Strings

- A Função strcmp():
  - Para comparar os caracteres contidos nas duas cadeias, é necessário utilizar uma função.
  - A função strcmp(), declarada no arquivo string.h, recebe dois endereços de strings e retorna um número inteiro, que indica a diferença numérica do primeiro caractere diferente da primeira cadeia em relação ao da segunda.

Outras funções de manipulação de Strings

- A Função strcmp():
  - Então se este número for:
    - < 0 string1 é menor que string2
    - = 0 string1 é igual a string2
    - > 0 string1 é maior que string2
  - Nesse contexto, “menor que” ou “maior que” indica que, se string1 e string2 forem colocadas em ordem alfabética (de dicionário), o que aparecer primeiro é menor.
  - Vamos reescrever o programa anterior corretamente:

Outras funções de manipulação de Strings

Prog21.c

```c
/* Prog21.c */
/* Mostra o uso de strcmp() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char resposta[]="BRANCO";
    char resp[40];

    printf("Qual é a cor do cavalo branco de Napoleão?");
    gets(resp);

    while (strcmp(resp,resposta) != 0)
    {
        printf("Resposta errada. Tente de novo.\n");
        gets(resp);
    }
    printf("Correto!\n");
    system("PAUSE");
    return 0;
}
```

Outras funções de manipulação de Strings

- A Função strcmp():
  - Veja um exemplo que imprime o valor retornado por strcmp() em várias situações:

Outras funções de manipulação de Strings

Prog22.c

```c
/* Prog22.c */
/* Testa a função strcmp() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    printf("%d\n", strcmp("A", "A"));
    printf("%d\n", strcmp("A", "B"));
    printf("%d\n", strcmp("B", "A"));
    printf("%d\n", strcmp("C", "A"));
    printf("%d\n", strcmp("casas", "casa"));

    system("PAUSE");
    return 0;
}
```

Outras funções de manipulação de Strings

- A Função strcpy():
  - A função strcpy(), declarada no arquivo string.h, recebe dois endereços de strings e copia o conteúdo da segunda na primeira.
  - Para ilustrar o seu uso, escreveremos uma função que apaga um caractere de uma cadeia de caracteres:

Outras funções de manipulação de Strings

Prog23.c

```c
/* Prog23.c */
/* Mostra o uso de strcpy() */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void strdel(char[], int); /*Protótipo */

int main()
{
    char str[] = "Carrta";

    printf("%s\n", str);
    strdel(str,2);
    printf("%s\n", str);

    system("PAUSE");
    return 0;
}
```

Outras funções de manipulação de Strings

```
20
21  /* Apaga um caractere de uma cadeia */
22  void strdel(char str[], int pos)
23  {
24      strcpy(str+pos, str+pos+1);
25  }
26
```

- A função strdel() move, em um espaço à esquerda todos os caracteres que estão à direita do que está sendo apagado.
- Para essa movimentação, a função utiliza strcpy();

As Funções strncat(), strncmp() e strncpy()

- Essas funções são semelhantes às que acabamos de analizar.
- A diferença é que elas trabalham com um número fixo de caracteres.
- Todas recebem um terceiro argumento indicando o número de caracteres a processar.

As Funções strncat(), strncmp() e strncpy()

- Por exemplo, a função strncat() concatena n caracteres da segunda cadeia na primeira; strncmp compara n caracteres das cadeias; e strcpy() copia n caracteres da segunda cadeia para a primeira.

Roteiro

- Strings
- As funções para manipulação de Strings
- Aritmética com endereços
- Outras funções para manipulação de Strings: strlen(); strcat(); strcmp(); strcpy(); strncat(); srtncmp(); strncpy()
- Matrizes de Strings.
- Um programa que imprime um cartão de Natal

Matrizes de Strings

- No decorrer deste capítulo, vimos vários exemplos de matrizes bidimensionais.
- Como uma string é uma matriz, uma matriz de strings é na realidade uma matriz de matrizes, ou uma matriz de duas dimensões.
- O nosso programa de exemplo é uma melhoria do programa que imprime os dias da semana que utilizamos no curso básico para estudar comandos de decisão.

Matrizes de Strings

- Ele solicita que seja digitada uma data e imprime o dia da semana.
- Observe como a matriz de strings é inicializada.
- Cada nome entre aspas é uma matriz de uma dimensão.

Matrizes de Strings

Prog24.c

```c
/* Prog24.c */
/* Imprime o dia da semana a partir de uma data */
/* Mostra o uso de uma matriz de strings */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <conio.h>  /* para getche() */

int dsemana(int, int, int); /*Protótipo */

int main()
{
    char diasemana[7][14]= {    "Domingo",
                                "Segunda-feira",
                                "Terça-feira",
                                "Quarta-feira",
                                "Quinta-feira",
                                "Sexta-feira",
                                "Sábado"
                           };
    int dia, mes, ano;
    const char ESC = 27;
    do
```

Matrizes de Strings

```
23      do
24      {
25          printf("Digite a data na forma dd mm aaaa: ");
26          scanf("%d%d%d", &dia, &mes, &ano);
27          printf("%s\n", diasemana [ dsemana(dia,mes,ano)] );
28          printf("ESC para terminar ou ENTER para recomecar\n");
29      }
30      while (getch() != ESC);
31
32      system("PAUSE");
33      return 0;
34  }
35
36  /* Encontra o dia da semana a partir de uma data
37   * Retorna 0 para domingo, 1 para segunda-feira etc.
38   */
39  int dsemana(int dia, int mes, int ano)
40  {
41      int dSemana = ano + dia + 3 * (mes - 1) - 1;
42      if( mes < 3)
43          ano--;
44      else
45          dSemana -= (int)(0.4*mes+2.3);
46      dSemana += (int)(ano/4) - (int)((ano/100 + 1)*0.75);
47      dSemana %= 7;
48      return dSemana;
49  }
50
```

Roteiro

- Strings
- As funções para manipulação de Strings
- Aritmética com endereços
- Outras funções para manipulação de Strings: strlen(); strcat(); strcmp(); strcpy(); strncat(); srtncmp(); strncpy()
- Matrizes de Strings.
- Um programa que imprime um cartão de Natal

Um programa que imprime um cartão de Natal

- O próximo programa usa as funções de manipulação de strings para imprimir um lindo cartão de Natal.
- Eis a listagem:

Um programa que imprime um cartão de Natal

Prog25.c

```
1  /* Prog25.c */
2  /* Imprime um cartão de natal */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <conio.h>  /* para getche() */
7
8  int main()
9  {
10     char Nome[81], Mensagem[39], sEsquerdo, sDireito, Contorno[42];
11     int Tamanho, Lado1=0, me=20, Direito, ns, i, fim, Lado, j;
12
13     printf("\nDestinatario: ");
14     gets(Nome);
15     printf("\nMensagem dentro da arvore: ");
16     gets(Mensagem);
17     Tamanho=strlen(Mensagem);
18     printf("\nSinal interno direito:");
19     sDireito=getche();
20     printf("\nSinal interno esquerdo:");
```

Um programa que imprime um cartão de Natal

```
20      printf("\nSinal interno esquerdo:");
21      sEsquerdo=getche();
22      Direito=38-Tamanho;
23      Contorno[0]='/';
24      ns=Direito/2;
25      for(i=1; i<=ns ; i++) Contorno[i]=sEsquerdo;
26      strcpy(&Contorno[i], Mensagem);
27      fim=strlen(Contorno);
28      for(i=fim; i < fim+ns ; i++) Contorno[i]=sDireito;
29      Contorno[i]='\\';
30      Contorno[i+1]='\0';
31      fim=strlen(Contorno);
32
33      /* Impressão da árvore */
34      printf("\n");
35      for(i=0; i < (80-strlen(Nome))/2; i++) printf(" ");
36      printf("%s\n\n\n",Nome);
37      for(i=1; i<=36; i++)
38      {
```

Um programa que imprime um cartão de Natal

```
37      for(i=1; i<=36; i++)
38      {
39          Lado=i-Lado1;
40          for(j=1; j <= me+20-Lado; j++) printf(" ");
41          for(j=0; j<Lado ; j++) printf("%c",Contorno[j]);
42          printf("%s\n",&Contorno[fim-Lado]);
43          if(!(i%4)) Lado1=Lado1+2;
44      }
45      for(i=0; i<me-1; i++) printf(" ");
46      printf("%c",'/');
47      for(i=0; i<20; i++) printf("%c",sEsquerdo);
48      for(i=0; i<20; i++) printf("%c",sDireito);
49      printf("%c\n",'\\');
50      for(i=0; i<4; i++)
51      {
```

Um programa que imprime um cartão de Natal

```
48      for(i=0; i<20; i++) printf("%c",sDireito);
49      printf("%c\n",'\\');
50      for(i=0; i<4; i++)
51      {
52          for(j=0; j<me+18; j++) printf(" ");
53          printf("| |\n");
54      }
55      printf("\n\n");
56      for(i=0; i<me+15; i++) printf(" ");
57      printf("FELIZ NATAL\n");
58      for(i=0; i<me+11; i++) printf(" ");
59      printf("E UM PROSPERO 2008!\n");
60      system("PAUSE");
61      return 0;
62  }
```

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;